Cost-Benefit Analysis of Agentic AI Design Patterns for E-Commerce

Ai-Ting, Lin
113427013
whereisaiting@gmail.com

Jia-Hua, Peng
114423056
jhpeng@g.ncu.edu.tw

Li-Feng, Chen
114423006
dowgojashan@gmail.com

Xin-Ping, Wu
114423067
hyperrrrrea7@gmail.com

Abstract

E-commerce customer service is increasingly adopting AI agent systems, but it remains unclear whether more complex agent designs generate sufficient marginal benefits to justify their additional costs. This study compares four Agentic AI design patterns — Single-Slot, ReAct, Reflection, and Plan-and-Execute — in an e-commerce customer service simulation. Using 150 in-sample (IS) cases and 50 out-of-sample (OOS) cases evaluated across three customer Personas (Polite, Adversarial, VIP), we conduct 2,400 evaluation conversations and assess each architecture on response quality, tool use, execution trajectory, efficiency, and cost-benefit performance. Results show that Plan-and-Execute achieves both the highest composite quality score and the lowest cost, producing positive marginal benefit (ΔMB) in both IS and OOS settings — the only architecture to do so. ReAct and Reflection both produce negative ΔMB, as their added complexity cannot be fully used within the capability range of llama3.1:8b. An OOS paradox is also observed: Plan-and-Execute's valid termination rate and Judge score diverge sharply under distribution shift, revealing that rule-based proxy metrics can decouple from semantic quality when the tool set hits its coverage limit. These findings suggest that architecture complexity alone does not guarantee service improvement, and that ΔMB — rather than single-metric evaluation — should guide architecture selection. For e-commerce platforms, Plan-and-Execute is recommended for mature intent coverage settings, while Single-Slot provides more stable cost protection in cold-start contexts.

1. Overview

E-commerce customer service has increasingly adopted AI-based systems to handle large volumes of repetitive and time-sensitive customer inquiries. Compared with human service representatives, chatbots can provide faster and simultaneous responses while reducing operational costs for platform e-tailers[1]. However, the value of AI in customer service is not determined only by automation efficiency.

Different service situations may require different levels of AI involvement, especially when customer inquiries vary in complexity, subjectivity, and the need for human judgment[1].

Recent advances in agentic AI have further shifted the discussion from simple chatbot adoption to the design of different AI agent frameworks[2]. These capabilities allow AI agents to handle more complex customer service workflows, but they also introduce additional challenges. More advanced agent designs may increase token costs, tool-use costs, response latency, operational complexity, and potential privacy or compliance risks.

Therefore, the key issue is no longer simply whether AI agents can support e-commerce customer service. Instead, the more important question is whether more complex agent frameworks generate sufficient marginal benefits to justify their additional costs and risks. Prior research suggests that AI systems should be evaluated not only by whether they are used, but also by how they are designed and whether the system design fits the task, user, and problem context. This perspective is especially important in commercial environments, where AI agents may influence customer experience, service decisions, and platform operations. Based on this concern, this study compares different AI agent frameworks in e-commerce customer service, including simple LLM responses, ReAct/tool-use agents, Reflection-based agents, and Plan-and-Execute agents. Rather than focusing only on final answer quality, this study evaluates whether these frameworks provide sufficient marginal benefits across response quality, task completion, tool use, cost, latency, hallucination, context tracking, and safety. By doing so, this study aims to clarify when a simple LLM response is sufficient and when more complex agent designs provide meaningful additional value.

2. Literature Review

2.1 AI in E-commerce Customer Service: From Automation to Service Strategy

E-commerce platforms have become important channels for firms to provide online customer service, and the quality of customer service interactions can influence customer satisfaction and customer retention. With the development of artificial intelligence and large language models, chatbots and AI-based service systems have been increasingly adopted in e-commerce customer service. Compared with human service representatives, chatbots can provide faster and simultaneous responses while reducing operational costs for platform e-tailers [1]. This makes AI particularly attractive for high-frequency and repetitive service tasks, such as order inquiries, delivery tracking, return policy explanation, and frequently asked questions.

Existing studies have compared different AI involvement strategies, including human-only, AI-only, and human–AI collaboration. Zhang et al. (2025) show that the effectiveness of these strategies depends on task complexity, service volume, and product margin. Specifically, AI-only service tends to be more cost-effective in low-volume and simple-task contexts, while human–AI collaboration performs better when service volume increases or inquiries become more complex [1].This finding suggests that AI service systems are not universally superior across all conditions. Instead, the value of AI depends on whether the service mode fits the characteristics of the task and the business context.

This strategic perspective is directly related to the concept of marginal benefit. A more advanced AI system may improve response quality, service speed, or task completion, but these improvements must be compared against the additional costs required to achieve them. In customer service operations, additional system complexity may increase token consumption, tool-use cost, response latency, maintenance difficulty, and privacy or compliance risks. Therefore, the central question is

not only whether AI can support e-commerce customer service, but whether more complex AI agent designs generate enough additional value to justify their additional costs.

2.2 LLM-based Agent Frameworks

Planning is one of the central capabilities that distinguishes AI agents from simple response generators. Huang et al. (2024) define autonomous agents as systems that accomplish tasks by perceiving the environment, planning, and executing actions [3]. They further argue that planning is one of the most critical capabilities of agents because it requires understanding, reasoning, and decision-making) [3].

Huang et al. (2024) categorize LLM-agent planning into five major directions: task decomposition, plan selection, external-module-aided planning, reflection, and memory [3]. This taxonomy is useful for understanding why different AI agent frameworks may produce different behaviors even when they receive the same customer service prompt. A basic LLM may directly generate an answer, while a planning-based agent may first decompose the task, select a plan, retrieve external information, reflect on potential errors, or use memory to maintain context across turns.

2.3 ReAct and Tool-use Agents

One important class of agent frameworks combines reasoning with action. ReAct, proposed by Yao et al. (2023), allows language models to generate reasoning traces and task-specific actions in an interleaved manner [4]. In this framework, reasoning traces help the model track and update action plans, while actions allow the model to interact with external sources such as knowledge bases or environments [4].

This approach is highly relevant to e-commerce customer service. Many customer service tasks require the system to retrieve or verify external information. A direct LLM response may produce a fluent answer, but it may not be grounded in actual transaction data. In contrast, a ReAct-style or tool-use agent can interact with external systems and use retrieved information to support its response.

ReAct also addresses limitations of reasoning-only approaches. Yao et al. (2023) argue that chain-of-thought reasoning can remain a static black box if it is not grounded in the external world, which may lead to hallucination and error propagation [4]. By allowing the model to interact with external environments, ReAct can reduce hallucination and improve interpretability in tasks such as question answering, fact verification, and interactive decision-making[4]. In the context of e-commerce customer service, this suggests that tool-use agents may improve factual grounding and task completion.

2.4 Reflection-based Agents

Reflection-based agents provide another technical approach to improving AI system performance. Shinn et al. (2023) propose Reflexion, a framework in which language agents learn not by updating model weights, but through linguistic feedback [5]. Reflexion agents verbally reflect on task feedback signals and store their reflective text in an episodic memory buffer to improve decision-making in subsequent trials [5].

This mechanism is useful for customer service because AI-generated responses may need to be checked for correctness, completeness, tone, and policy compliance before being sent to users. A reflection-based agent may generate an initial answer, review potential weaknesses, identify missing information, and revise the final response. In e-commerce customer service, this may help reduce careless errors, incomplete refund explanations, unclear delivery updates, or unsupported claims about company policy.

However, reflection also illustrates why more complex agent designs do not always guarantee higher marginal benefits. Reflexion depends on feedback signals, self-evaluation, and memory. Although these mechanisms may improve performance, they also introduce additional reasoning steps. Shinn et al. (2023) note that Reflexion relies on the self-evaluation capability of the LLM or heuristic feedback and does not provide a formal guarantee of success[5]. This limitation is important in customer service because an agent may repeatedly revise or question its own answer without reaching a stable final response.

2.5 Plan-and-Execute Agents

Plan-and-Execute agents improve the controllability of AI agents by separating the reasoning process into two stages: planning and execution. Instead of directly generating a final response, the agent first decomposes the user request into smaller subtasks and then executes them step by step. This approach is consistent with the broader literature on LLM-agent planning, which identifies task decomposition, plan selection, external modules, reflection, and memory as key mechanisms for handling complex tasks[3]. It is also related to Plan-and-Solve prompting, which first devises a plan to divide a task into subtasks and then solves them according to the plan, reducing missing-step errors in multi-step reasoning.

In e-commerce customer service, Plan-and-Execute is useful because many customer requests require sequential verification, such as identifying customer intent, checking order information, reviewing return policies, and generating a compliant response. Compared with ReAct, which interleaves reasoning and actions during task execution, Plan-and-Execute provides a clearer separation between planning and execution, making the workflow easier to inspect and control [4]. However, this framework also has limitations. If the initial plan is incomplete or incorrect, the execution process may still follow the wrong direction. Prior research also suggests that LLMs may struggle with reliable long-horizon planning without external planning support (Liu et al., 2023). Therefore, Plan-and-Execute agents should be evaluated not only by response quality, but also by whether their additional planning steps justify the extra cost, latency, and operational complexity.

2.6 Research Gap

Prior studies have shown that AI-only, human-only, and human–AI collaboration strategies in e-commerce customer service may produce different outcomes depending on task complexity, service volume, and product margin [1]. Meanwhile, research on Agentic AI suggests that customer service agents can incorporate task planning, memory, tool use, and knowledge retrieval, but they still face limitations in evaluation methods, failure analysis, and cost-benefit modeling[2]. In addition, technical studies on LLM agents indicate that different agent frameworks vary in planning, tool use, reflection, memory, and execution control, which may lead to different levels of service quality, cost, and risk. However, relatively few studies have directly compared the marginal benefits of different AI agent frameworks within the same e-commerce customer service setting. In particular, it remains unclear whether more complex agent designs can generate sufficient improvements in response quality, task completion, clarity, and safety to justify their additional token costs, tool-use costs, response latency, and operational complexity.

Moreover, recent research suggests that AI agent evaluation should not focus only on final answers, but should also consider tool-use trajectories, multi-turn interactions, safety risks, and LLM-as-judge-assisted assessment. Therefore, this study compares the performance of different agent frameworks

under similar customer service prompts and evaluates whether they provide sufficient marginal benefits in terms of response quality, task completion, tool use, cost, latency, hallucination, context tracking, and safety.

3. Methodologies

3.1 Overview

This study aims to compare the cost-effectiveness of four Agentic AI design patterns—Single-Slot, ReAct, Reflection, and Plan-and-Execute—in the context of e-commerce customer service. The evaluation framework encompasses dataset construction, architecture implementation, multi-dimensional scoring metrics, and an integrated cost-benefit model. The experimental design prioritizes reproducibility and comprehensive multi-dimensional coverage.

The experiment is centered on 150 e-commerce customer service cases spanning three difficulty levels—Easy, Medium, and Hard—and subjected to full evaluation across three customer Personas (Polite, Adversarial, and VIP), yielding a total of 1,800 evaluation records. An independently constructed set of 50 Out-of-Sample (OOS) cases was additionally employed to validate each architecture's cross-context generalizability, producing 600 OOS evaluation records.

To confirm that performance differences across architectures stem from architectural design rather than specific customer input styles, this study employs Multi-Persona stress testing as a Prompt Sensitivity Analysis under the ProSA framework (see Section 3.6).

3.2 Agentic AI Architecture Designs

3.2.1 Baseline Architecture Configuration

All four architectures share a common model configuration to ensure fair inter-architecture comparisons.
Although the prompt structures vary across the four architectures, all architectures adhere to the following four core principles:

- Identity-First: An Order ID or email address must be present in the conversation history. If no ID is provided, the agent immediately requests one and refrains from invoking any tool calls.
- Observation Ground Truth: Tool results are injected by the system; the agent must not predict or fabricate tool return values. Prompts explicitly prohibit hallucinated statements such as "I've checked your order"

TABLE 1: Four Baseline Architecture .

Component	Model	Deployment	Purpose
CSR Agent	Llama 3.1:8b	Ollama (Local)	Generate customer service responses and reasoning
Customer Agent	Llama 3.1:8b	Ollama (Local)	Simulate diverse customer behaviors
LLM-as-a-Judge	Gemini 3.1 Flash-Lite	Google API	Multi-dimensional response quality scoring

TABLE 2: Architecture Comparison Overview .

Dimension	Single-Slot	ReAct	Reflection	Plan-and-Execute
Explicit Reasoning	✗	✓ (Thought)	✓ (Draft + Reflection)	✓ (Plan)
Iteration Capability	None	Up to 5	Up to 5	Up to 5
Self-Correction	None	Moderate (Observation-based)	High (Explicit reflection stage)	Moderate (Plan adjustment)
Anti-Hallucination Mechanism	Prompt-level	Prompt-level	Prompt + Reflection level	Prompt + Plan validation
Expected Token Consumption	Lowest	Moderate	Highest	Moderate
Expected LLM Calls	1	2–5	2–5	2–5

- prior to actual tool execution.
- Consent-First: Irreversible operations such as cancellations and refunds must be executed only upon explicit customer
- consent; tools must not be invoked based on inferred intent.
- One Tool Per Turn: Only one tool may be called per iteration cycle. Execution halts immediately following the Action output, awaiting system injection of the Observation.

3.2.2 Four Architecture Design

1. Architecture 1: Single-Slot
   Single-Slot serves as the zero-shot baseline architecture, generating responses via a single LLM call without intermediate reasoning or iterative steps. The prompt employs a three-option decision framework: (A) if an Order ID is present in the conversation history, directly output a tool call; (B) if no Order ID exists, politely request one from the customer; (C) upon receiving a tool Observation, invoke the appropriate follow-up tool or output the final response. The primary advantage of this design is its minimal computational cost; however, it lacks the capacity for reasoning and self-correction in complex scenarios.

2. Architecture 2: ReAct (Reasoning and Acting)[4]
   The ReAct architecture interleaves Thought (internal reasoning) and Action (tool invocation). In each iteration, the agent first explicitly outputs its reasoning process before deciding to invoke a tool or produce a final response. The system subsequently injects an Observation for the next iteration, with a maximum of five iterations. Prompts enforce plain-text tag formatting and mandate halting after the Action line; loop control and tool

execution are managed by an external runner. Compared to Single-Slot, ReAct provides the ability to dynamically adjust reasoning paths based on Observations and represents the standard Agentic pattern widely adopted in industry.

3. Architecture 3: Reflection (Three-Stage Deliberation)[5]
   The Reflection architecture extends ReAct by introducing a three-stage loop: Initial Draft, Reflection, Final Response. The Reflection stage explicitly audits the initial draft for violations of the consent rule, hallucinated tool results, and the rationality of tool call sequencing, then outputs a corrected result in the Final Response. To reduce unnecessary token consumption at conversation close, the architecture automatically simplifies the reflection process upon detecting farewell keywords (e.g., "thank you", "bye"). This architecture exhibits the strongest anti-hallucination capability, but also incurs the highest token consumption per iteration.

4. Architecture 4: Plan-and-Execute[6]
   The Plan-and-Execute architecture divides each iteration into two explicit stages: Plan (decomposing the current task into an ordered sequence of steps) and Execution (carrying out the current step in the plan). Unlike ReAct's step-by-step iteration, this architecture outputs a complete plan at each reasoning step, ensuring that every decision is made within a global context. Prompts explicitly define operational constraints for each order state (e.g., cancel_order is disallowed when status = Shipped) and require the agent to dynamically update the execution plan upon receiving an Observation. This design provides greater decision transparency and is well-suited for complex tasks requiring structured reasoning.

3.3 Dataset Construction

3.3.1 In-Sample Dataset

The primary evaluation in this study draws upon the Bitext Gen AI Chatbot Customer Support Dataset (Kaggle, 2023). This dataset is a hybrid synthetic corpus generated using NLP/NLG techniques and reviewed by linguists, providing well-structured single-turn customer service question-answer pairs suitable for controlled agent evaluation experiments.

3.3.1.1 Fact Sheet Construction

To establish an objective benchmark for agent system evaluation, this study preprocessed the raw Bitext corpus using Python scripts to construct a dedicated structured Fact Sheet. The pipeline ingests the raw dataset, applies programmatic data cleaning and environmental variable augmentation (e.g., order status, refund eligibility, shipment tracking numbers), and produces the objective Fact Sheet used as factual ground truth throughout evaluation. All data are either generated through programmatic rules or inherited from the original dataset, strictly excluding LLM-introduced hallucination risk.
The resulting Fact Sheet serves as the benchmark for rigorously examining agent performance across the core dimensions of Intent Recognition, Response Quality, Hallucination Control, and Task Completion.

3.3.1.2 Evaluation Case Design and Difficulty Stratification

A total of 150 independent evaluation cases were sampled in this study. To comprehensively assess agent reasoning and execution capabilities across diverse scenarios, cases were categorized into three difficulty levels based on the number of tool calls

required and the complexity of the decision logic involved:

Level 1: Easy — Single Query Type

- Task Characteristics: Requires only a single tool call to retrieve existing information; does not involve complex downstream decision logic.
- Case Range: 50 cases (CASE_001 – CASE_050).
- Representative Tasks: Retrieving refund policies, querying payment methods, inquiring about shipping options.

Level 2: Medium — Multi-Step Decision Type

- Task Characteristics: Requires 2 – 3 tool calls; the agent must verify multiple information fields before making a status determination and executing operations.
- Case Range: 50 cases (CASE_051 – CASE_100).
- Representative Tasks: Canceling orders, modifying delivery addresses.

Level 3: Hard — Complex Logic Type

- Task Characteristics: Requires more than 3 tool calls; involves complex business rule adjudication, multi-factor identity verification, or handling hidden intents (Hidden Slots) within the conversation.
- Case Range: 50 cases (CASE_101 – CASE_150).
- Representative Tasks: Modifying order contents (e.g., removing specific items), cross-system refund status tracking.

3.3.2 Out-of-Sample Datase

To validate each architecture's generalizability to unseen cases, this study additionally employs the Customer Support Conversations Dataset (Kaggle) as the Out-of-Sample (OOS) evaluation set. This dataset consists of high-quality synthetic data simulating real-world multi-turn conversational logic across domains such as e-commerce, banking, and SaaS. It provides rich annotation and is specifically designed for LLM evaluation and Agentic Workflow stress testing, with no associated privacy risks.

The dataset comprises approximately 25,000 complete multi-turn conversations in a Multi-turn format. Each conversation_id contains multiple messages alternately generated by Customer and Agent roles, faithfully reflecting the interactive process of problem resolution.

TABLE 3: Generalization Test Levels and Evaluation Scope.

Generalization Test Level	Description
Overlapping Intent Generalization	6 intents already seen in In-Sample (e.g., track_order, get_refund); tests stability on familiar task types
Novel Intent Generalization	9 entirely new intents (e.g., track_return, fraud_dispute, missing_item); tests the architecture's ability to handle unseen task types
Novel Category Generalization	Two new categories, RETURNS and PRODUCT, testing cross-domain generalization

3.3.2.1 OOS Case Set Design

This study independently constructed 50 OOS cases (Case IDs: OOS_001–OOS_050), stored in data/oos_fact_sheets.json, with no overlap with the In-Sample case bank. The difficulty distribution is approximately balanced (Easy: 15 cases; Medium: 17 cases; Hard: 18 cases).
The OOS cases span 15 intents, of which 9 are novel intents not present in the In-Sample set, along with 2 entirely new categories (RETURNS and PRODUCT). The design aims to simultaneously test generalizability at two levels.
The OOS evaluation is conducted over 50 cases × 4 architectures × 3 Personas = 600 records. Comparisons of S_Agent, PSS, and architecture ranking consistency between the In-Sample and OOS sets are used to validate the external validity of the study's conclusions.
The evaluation proceeds in three phases: Phase 1 (Pilot, 15 cases × 3 runs each) validates prompt stability and metric correctness; Phase 2 (full In-Sample, 150 × 4 × 3 = 1,800 records) produces the primary data for architecture comparison, PSS, and

ΔMB analyses; Phase 3 (OOS, 50 × 4 × 3 = 600 records) validates generalizability on unseen cases (see Section 3.3.2).

3.4 Evaluation Framework

This study adopts case-level evaluation to compare four customer service agent architectures: Single-slot, ReAct, Reflection, and PlanExecute. Because the performance of Agentic AI depends not only on the final response, but also on tool selection, argument correctness, tool-result usage, execution trajectory, and resource consumption, this study does not use a single LLM-as-a-Judge score as the overall evaluation criterion. Instead, the evaluation framework is divided into Outcome Quality, Process Quality, Efficiency, and Safety Gate, with cost-benefit performance analyzed separately.
The evaluation framework is grounded in prior work on e-commerce customer service and LLM-agent evaluation. E-commerce studies show that AI service value depends on response quality, task complexity, speed, and operational cost[1], while agentic AI research emphasizes planning, tool use, and autonomous action beyond traditional chatbot responses[2, 7]. Recent agent evaluation studies further argue that LLM agents should be assessed not only by final answers, but also by tool use, reasoning trajectories, multi-turn interaction, safety, and cost-efficiency [8-10]). Therefore, this study adopts turn-level logging with case-level scoring and evaluates agents across outcome quality, process quality, efficiency, and safety, while additionally examining cost-benefit performance.

3.4.1 Outcome Quality

Outcome Quality evaluates whether the final customer service response is effective and grounded. It consists of S_AnswerQuality and S_Grounding:

S_Outcome = 0.60*S_AnswerQuality + 0.40*S_Grounding

S_AnswerQuality is evaluated by an LLM-as-a-Judge and focuses only on the quality of the final response. It does not evaluate tool-use process, cost, or safety gate. The Judge scores three sub-dimensions on a 1–5 scale: S_Resolution, S_Completeness, and S_Tone. To integrate these scores with other 0–100 scale metrics, each sub-score is first converted from the 1–5 scale to a 0–100 scale:

S_{d,100} = (S_d - 1) / 4 * 100

where S_d denotes the original 1–5 score for a given sub-dimension, and S_{d,100} denotes the

TABLE 4: Weighted Evaluation Scheme for AI Agent Performance.

Component	Weight	Rationale
S_Outcome	0.50	The core goal of customer service is to correctly resolve the customer's issue.
S_Tool	0.20	Tool use is a key differentiator of Agentic AI.
S_Trajectory	0.10	This evaluates workflow reasonableness while avoiding over-penalizing alternative valid paths.
S_Efficiency	0.20	This reflects time or resource efficiency in practical deployment.

converted 0–100 score. The converted scores are then weighted to calculate S_AnswerQuality:

S_AnswerQuality = 0.50*S_{Resolution,100} + 0.30*S_{Completeness,100} + 0.20*S_{Tone,100}

Here, S_Resolution evaluates whether the customer's core need is correctly addressed, S_Completeness evaluates whether the response provides sufficient information and next steps, and S_Tone evaluates whether the tone is professional, clear, and appropriate for a customer service context.

S_Grounding is computed using rule-based checks. It examines whether the final answer is consistent with the ground truth in the case fact sheet, order facts obtainable through tools, and policy constraints in policy_ground_truth.json. The current implementation checks order ID, order status, amount, unauthorized cancellation/refund claims, and policy forbidden claims / forbidden conditions. If no grounding check is applicable, the score defaults to 100. Otherwise, the score is calculated based on the pass rate of applicable checks:

S_Grounding = 100 * passed_grounding_checks / total_applicable_checks

3.4.2 Process Quality

Process Quality evaluates whether the agent uses tools and advances the task in a reasonable way. It consists of S_Tool and S_Trajectory.

S_Tool evaluates tool selection, argument correctness, and tool-result usage:

S_Tool = 100*(0.50*ToolF1 + 0.25*ArgumentCorrectness + 0.25*ResultUsage)

ToolF1 compares the tools actually used with the expected_tools specified in the fact sheet, while allowing valid_alternative_tools as acceptable alternative paths. ArgumentCorrectness checks whether tool arguments such as order ID or email are correct. ResultUsage checks whether tool results are reflected in the subsequent response.

S_Trajectory is currently computed through rule-based checks. These include whether the agent queries the order before executing cancellation or refund actions, whether repeated tool-use loops occur, whether there are excessive redundant tool calls, and whether sufficient identifying information is obtained before tool calls.

S_Trajectory = 100 * passed_trajectory_checks / total_trajectory_checks

TABLE 5: Evaluation Metrics and Data Sources for AI Agent Performance.

Dimension	Metric	Method	Data Source	Included in S_Agent
Task Status	final_resolution	rule/log-based	log metadata	No
Outcome	S_AnswerQuality	LLM Judge	conversation + fact sheet	Yes
Outcome	S_Grounding	rule-based	final answer + fact sheet + policy GT	Yes
Process	S_Tool	rule-based	full trace + fact sheet	Yes
Process	S_Trajectory	rule-based	conversation + tool trace	Yes
Efficiency	S_Efficiency	log-based	execution time / tokens	Yes
Safety	I_fatal	rule-based gate	final answer + policy/tool status	Cap only
Cost-benefit	ProxyCost, NetValue, Delta_MB	formula-based	token/call/log metrics	Separate analysis

3.4.3 Efficiency

S_Efficiency measures execution efficiency relative to the Single-slot baseline under the same difficulty level:

S_Efficiency(a,i) = min(100, baseline_d / cost_a,i * 100)

Here, cost_a,i refers to execution_seconds when available, and grand_total_tokens otherwise. If execution_seconds is available in the log, the median execution time of Single-slot cases at the same difficulty level is used as the baseline. If older logs do not contain execution time, grand_total_tokens is used as a proxy. This design avoids fully duplicating the later ProxyCost measure while preserving comparability with earlier experimental logs.

3.4.4 Safety and Compliance Gate

I_fatal is a safety and compliance gate rather than a general quality score. If a major violation is detected, such as an unauthorized claim that a refund has been processed, an unauthorized claim that an order has been cancelled, a fabricated refund reason, or an explicit violation of the cancellation policy, then I_fatal = 1. This study uses a score cap rather than hard zero:

If I_fatal = 0, then S_Agent = S_raw.

If I_fatal = 1, then S_Agent = min(S_raw, 40).

This design preserves the safety constraint while still allowing the study to analyze differences in the original performance of fatal cases.

3.4.5 Overall Agent Score

The current implementation adopts customer-centric weighting, placing greater emphasis on final customer service outcomes:

S_raw = 0.50*S_Outcome + 0.20*S_Tool + 0.10*S_Trajectory + 0.20*S_Efficiency

S_Outcome receives the highest weight because the core objective of customer service is to correctly

resolve the customer's issue. S_Tool and S_Trajectory capture tool-use quality and workflow quality in Agentic AI. S_Efficiency reflects practical response time or token-proxy cost.

3.5 Cost-Benefit Metric

In addition to quality scores, this study uses ProxyCost, NetValue, and Delta_MB to evaluate the cost-benefit performance of different architectures.

3.5.1 Proxy Cost

ProxyCost normalizes and weights token usage, LLM calls, and tool calls:

ProxyCost = 100*(0.5*TokenNorm + 0.3*LLMCallNorm + 0.2*ToolCallNorm)

In the current implementation, TokenNorm uses usage_summary.grand_total_tokens, LLMCallNorm is approximated by the number of conversation turns, and ToolCallNorm is derived from parseable Action: tool calls in the full trace; therefore, ProxyCost should be interpreted as a relative cost proxy rather than an exact monetary cost.

3.5.2 Net Value

Cost-adjusted net value is defined as follows:

NetValue(a,i) = V_i * S_Agent(a,i)/100 - lambda * ProxyCost(a,i)

where a denotes the Agent architecture and i denotes the case. V_i is the task value. It is first read from v_i in the fact sheet; if this field is missing, it falls back to the difficulty-based setting: Easy = 1, Medium = 2, and Hard = 3. S_Agent(a,i) is the overall performance score of architecture a on case i, while ProxyCost(a,i) represents the relative resource consumption of that case.

This study sets lambda = 0.01 as a scale-calibration parameter for the cost penalty term. Since V_i * S_Agent/100 is approximately in the range of 0–3, while ProxyCost is normalized to 0–100, directly subtracting ProxyCost would cause the cost term to dominate NetValue. Therefore, lambda = 0.01 scales the cost penalty to approximately 0–1, allowing it to influence the cost-benefit evaluation on a comparable scale with the task-value term. This value is not treated as the only valid setting; its reasonableness and influence on architecture ranking will be further examined through sensitivity analysis in Section 4.3.

TABLE 6: Customer Persona Profiles and Behavioral Characteristics.

Persona	Role	Core Behavioral Characteristics
Polite	Baseline	Measured tone, high cooperativeness; clearly articulates needs and follows standard procedures
Adversarial	Stress-Test Group	Impatient, confrontational, and emotionally charged; may challenge policies or demand compensation beyond established rules
VIP	Privilege-Expectation Group	Expects prioritized handling and special treatment; exhibits impatience toward wait times or standard procedures

This formula is used to determine whether a more complex Agent architecture can offset its additional

cost through quality improvement. If an architecture improves S_Agent but substantially increases token usage, LLM calls, or tool calls, NetValue reflects whether the quality gain is sufficient to compensate for the additional resource consumption.

3.5.3 Marginal Benefit

Using Single-slot as the baseline, this study calculates the marginal benefit of other architectures as follows:

Delta_MB(a,i) = NetValue(a,i) - NetValue(Single-slot,i)

If Delta_MB > 0, the architecture has higher cost-adjusted benefit than Single-slot for that case. If Delta_MB < 0, the additional reasoning or tool-use cost is not offset by the quality improvement.

3.6 Multi-Persona Prompt Sensitivity Analysis

3.6.1 Experimental Design

Each customer Agent is defined by an independent system prompt specifying its character traits and behavioral tendencies; all agents are driven by the same task script (Fact Sheet). The three Personas collectively represent the most characteristic spectrum of customer behaviors in e-commerce customer service scenarios.

3.6.2 PSS Formula

During the formal evaluation phase, each case–architecture combination is executed once under each of the three Personas, covering all difficulty levels (Easy, Medium, Hard), yielding a total of 1,800 evaluation records. The three Personas collectively constitute the prompt variant set P = {Polite, Adversarial, VIP} under the ProSA framework[1], and the performance gap between each pair of Personas constitutes one comparison sample.

Based on C(3,2) = 3 Persona pairs, the instance-level sensitivity score Si is defined as follows[11]:

$$S_i = \frac{\sum p_a, p_b \in P, p_{a \neq} p_b |Y(p_a)_i - Y(p_b)_i|}{C(|P|, 2)}$$

where Y(p)i denotes the SAnswer_Quality scor for case i under Persona p (LLM-as-a-Judge rating, normalized to [0,1]). Three comparison pairs are thus formed:

$$|Y_{\text{Polite}} - Y_{\text{Adversarial}}|, \quad |Y_{\text{Polite}} - Y_{VIP}|,$$

$$|Y_{\text{Adversarial}} - Y_{VIP}|$$

3.6.3 PromptSensiScore (PSS)[11]

The overall PSS is the mean of instance-level scores across all cases:

$$SS = \frac{1}{N} \sum_{i=1}^{N} S_i$$

where N denotes the total number of evaluation cases. In this study, the variable Y(p) specifically adopts the Answer Quality score (SAnswer_Quality) assessed by LLM-as-a-Judge. The rationale for this design choice is that different customer Personas primarily perturb the language model through variations in tone, intensity of pressure, and negotiation strategy, and the dimension most directly reflecting this perturbation effect is the completeness, decision accuracy, and service tone of the final generated response. Therefore, computing PSS with a focus on SAnswer Quality more precisely captures the

model's sensitivity to input style at the semantic and reasoning levels, and avoids dilution by rule-driven tool execution scores.

A lower PSS value indicates greater architectural stability with respect to variations in customer behavioral style and tone; conversely, a higher PSS value signifies that the architecture's output quality is highly susceptible to input style perturbations, posing elevated risks to response consistency in real-world commercial deployment scenarios. Within this study's evaluation framework, PSS serves as a diagnostic control metric: an architecture with a notably elevated PSS is considered inherently unstable, and high performance scores must be interpreted with caution.

4. Results

4.1 In-Sample Evaluation Results (1,800 Conversations)

PlanExecute stands out with S_Agent = 83.07. Its lead comes from balanced performance across all dimensions rather than one high score. ReAct has the highest S_Outcome (81.8), but its S_Tool of only 47.7 and S_Traj of 63.1 pull its total score down. Reflection's S_Efficiency of only 23.6 shows a heavy penalty for token usage (average 75,105 tokens, which is 4.4 times higher than PlanExecute). PlanExecute also shows the smallest S_Agent gap across Personas (1.65 points).

TABLE 7: In-Sample Performance by Architecture (Average across Three Personas).

Architecture	Valid Termination Rate	S_Outcome	S_Tool	S_Traj	S_Eff	**S_Agent**	Judge Score
PlanExecute	98.2%	79.0	**72.8**	**99.4**	97.0	**83.07**	64.2
Single-slot	59.5%	74.4	71.7	**99.9**	85.2	78.53	57.4
ReAct	**99.1%**	**81.8**	47.7	63.1	**98.0**	76.37	**69.8**
Reflection	65.4%	75.2	67.8	**100.0**	23.6	65.80	60.7

TABLE 8: OOS Performance and Drop from IS.

Architecture	OOS Valid Termination Rate	**OOS S_Agent**	OOS Judge	IS→OOS S_Agent Drop
PlanExecute	**96%**	**73.2**	27.3	-9.9
Single-slot	47%	70.0	30.4	-8.5
ReAct	76%	65.8	**35.3**	**-10.6**
Reflection	80%	60.8	33.5	-5.0

TABLE 9: ProxyCost / NetValue / ΔMB (Average across Three Personas).

Architecture	Token IS	Token OOS	ProxyCost IS	NetValue IS	ΔMB IS	NetValue OOS	ΔMB OOS
PlanExecute	**16,999**	**21,222**	**20.83**	**1.429**	**+0.162**	**0.512**	**+0.125**
Single-slot	24,086	29,856	28.70	1.247	0 (baseline)	0.387	0 (baseline)
ReAct	20,831	21,367	30.30	1.202	-0.059	0.363	-0.023
Reflection	75,105	72,928	33.90	0.938	-0.267	0.274	-0.113

4.2 Out-of-Sample Generalization Evaluation (600 Conversations)

The OOS dataset contains 9 new intent types not seen in the IS set (such as track_return, missing_item, fraud_dispute), used to test how well each architecture handles unknown tasks.
The overall ranking (PE > SS > ReAct > Ref) stays the same in OOS, but ReAct has the largest drop (-10.6). Its Polite Persona LOOP_FAILURE rate reaches 42%, meaning the iterative loop cannot stop when it faces unknown intents. Judge scores fall sharply to 22–38 (compared to IS architecture averages of 57–70), showing that the current tool set cannot handle the new intents. PlanExecute shows an OOS paradox: it has the highest valid termination rate (96%) but the lowest Judge score (27.3). The termination rule marks the conversation as done, but the actual response quality does not meet customer needs.

4.3 Cost-Benefit Analysis

with positive ΔMB in both IS and OOS. Its low-cost advantage holds well in OOS (ΔMB drops only from +0.162 to +0.125). In OOS, all architectures see NetValue fall by about 60% (IS: 0.94 – 1.43 → OOS: 0.27 – 0.51). The drop in quality has a much larger effect than the change in cost. To check whether the choice of $\lambda = 0.01$ affects the conclusions, this study scans λ across its full range ($\lambda \in [0.001, 0.5]$) and recalculates ΔMB for each architecture while keeping S_Agent and ProxyCost fixed.

TABLE 10:ΔMB by Architecture at Key λ Values (IS)

λ	PlanExecute	ReAct	Reflection
0.001	+0.099	−0.035	−0.238
0.01 (adopted)	**+0.162**	**−0.059**	**−0.267**
0.05	+0.444	−0.169	−0.396
0.1	+0.797	−0.307	−0.558

In IS, the ranking does not change at any λ value. The reason is that PlanExecute has both the highest quality (S_Agent 83.1) and the lowest cost (ProxyCost 20.8), so it dominates Single-slot on both dimensions at once. This means the ranking does not depend on the value of λ.
In OOS, a small local reversal appears: ReAct's ΔMB becomes positive near $\lambda \approx 0.03$, because Single-slot's OOS ProxyCost (31.3) is higher than ReAct's (29.5). When the cost penalty is large enough, the cost gap reverses the quality gap.
At $\lambda = 0.01$, this reversal has not yet happened (ReAct OOS $\Delta MB = -0.023$), so the λ value used in this study falls within the stable range before the reversal point. Reflection has negative ΔMB at all tested λ values, so its conclusion is not affected by the choice of λ.

4.4 Persona Sensitivity (PSS) and Ranking Stability

PlanExecute ranks first across all 969 valid weight combinations, showing that the study's conclusions are fully stable regardless of how the metrics are weighted. PlanExecute also achieves the lowest PSS in IS (17.96), showing consistent and stable output across all Personas. In OOS, ReAct's PSS drops to the lowest (17.20), reversing its IS rank from second to first by a margin of 1.13 points. PlanExecute's OOS PSS (18.33) is nearly the same as its IS value (17.96), showing that its stability has not gotten worse in OOS, while ReAct's gap across Personas does narrow in OOS. ReAct's lower OOS PSS reflects the fact that unknown intents create the same difficulty for all Personas: in IS there were 47 cases with PSS > 30 (where Persona behavior differences were large enough to affect the outcome), but in OOS only 5 such cases remain.

TABLE 11: ProSA PSS and S_Agent Weight Sensitivity.

Architecture	IS PSS	OOS PSS	IS Rank 1 (969 combinations)	OOS Rank 1
PlanExecute	**17.96**	18.33	**100%**	**99.9%**
ReAct	20.27	**17.20**	0%	0%
Single-slot	25.08	20.87	0%	0.1%
Reflection	28.70	20.13	0%	0%

Unknown intents create the same cognitive barrier for all Personas, so the gap between Personas becomes much smaller.

5. Analysis and Discussion

5.1 Quality Analysis

PlanExecute's lead comes from splitting the task into planning and execution, which lowers the cognitive load on the model. Traditional ReAct requires the model to handle reasoning, tool calls, and observation use all at once in each loop. PlanExecute instead breaks the task into two separate stages: the planning stage turns the customer's intent into a clear list of actions, and the execution stage only needs to complete one pre-defined action at a time. For llama3.1:8b, completing a single known action is much easier than doing multi-step reasoning on the fly, and this shows directly in S_Tool (highest overall) and S_Traj's top scores, as well as the smallest S_Agent gap across Personas.

The other three architectures each have their own quality problems rooted in their design. ReAct's tool call quality is notably low. The root cause is that the Thought-Action format places a demand on the model that cannot be removed — llama3.1:8b struggles to keep the format stable when context builds up across multiple turns, and tends to produce good reasoning in the Thought step but then use wrong parameters in the Action step, which also pulls S_Traj down. Reflection's problem is not in tool calls but in the gap between what the self-correction design assumes and what the model can actually do. The three-stage design assumes the model can spot and precisely fix problems in its draft, but in practice llama3.1:8b tends to add more content rather than making targeted fixes, so quality gains are not reflected in proportion in S_Agent, and it ends with the lowest S_Agent overall. Single-slot ranks second in IS, showing that a single-round design can still produce complete responses when the tool set is sufficient, but the lack of any correction step creates a structural ceiling on complex multi-step tasks.

In IS, the S_Agent ranking and the Judge score ranking do not match: ReAct has the highest Judge score but ranks third in S_Agent, while PlanExecute ranks first in S_Agent but only second in Judge score (see Table 1). This gap comes from the fact that the two measures capture different aspects of quality. S_Agent measures process quality, including verifiable execution metrics like tool call accuracy (S_Tool) and conversation format discipline (S_Traj). Judge score measures output quality, where an LLM evaluator judges how helpful and natural the final response sounds from the customer's point of view. ReAct's multi-turn reasoning builds up rich content that makes the final response detailed and clear, so it scores well on output quality. PlanExecute's execution steps are precise and correct, but the responses tend to be shorter and more structured, which leads to a

slightly lower Judge score. This gap shows that if Judge score were the only metric, the real risks from ReAct's tool call failures would be hidden, and the multi-dimensional design of S_Agent is specifically meant to avoid this kind of blind spot. OOS performance differences show how sensitive each architecture's stopping mechanism is to gaps in tool coverage. PlanExecute's stopping mechanism does not depend on a success signal from the tools. After the planning stage creates a list of actions, the execution stage simply works through the list and ends the conversation when done, without needing a successful tool result as a stop condition. This means PlanExecute can finish the conversation in a predictable way even when the tool set cannot handle a new intent, avoiding the endless loop problem. ReAct works in the opposite way: it uses a successful observation as its exit signal, and when the tool set cannot cover an OOS intent, the condition needed to stop the loop is structurally unreachable. The loop keeps running until the repeat-detection mechanism triggers a LOOP_FAILURE (reaching 42% for ReAct's Polite Persona in OOS). Single-slot outperforms ReAct in OOS because a single-round architecture outputs a general response and ends in the first round when it faces an unknown intent, skipping the process where an iterative architecture accumulates penalty with each failed attempt. This shows that when tool coverage is uncertain, failing quickly and predictably scores better overall than trying repeatedly and failing. Reflection has the smallest OOS drop but the lowest absolute score, and the small drop comes from its already low IS starting point rather than from strong generalization — it should not be read as a stability advantage.

The PSS analysis raises a theoretical question: in IS, simple cases have higher PSS than hard cases, which is the opposite of what ProSA (Zhuo et al., 2024) predicts. The real reason is not task difficulty but what this study calls the "success boundary effect." Hard cases are close to failure for all Personas, so scores are clustered in the low range and the gap between Personas is naturally compressed. Simple cases have the potential for success, which means Persona behavior differences are large enough to change the outcome and make the score gap bigger. It is also worth noting that part of the difference from ProSA comes from a difference in how perturbation is applied: ProSA uses prompt rewrites that mainly affect semantic understanding, while this study uses Persona behavior styles that introduce a challenging dynamic across multiple turns. The two approaches trigger different sensitivity mechanisms and should not be directly compared. In OOS, ReAct's lower PSS is a genuine reflection of how the architecture behaves: unknown intents create the same cognitive barrier for all Personas. In IS there were 47 cases with PSS > 30, but in OOS only 5 remain, and the success boundary effect almost disappears.

5.2 Cost Analysis

The cost ranking from lowest to highest is PlanExecute < Single-slot < ReAct < Reflection (see Table 3), and it is driven by three separate mechanisms. PlanExecute has the lowest ProxyCost, and it is surprisingly lower than Single-slot. The reason is that its structured division of work keeps each output short: the planning step only produces a short list of actions, and each execution step only needs to complete one pre-defined action. This keeps token usage 29% lower than Single-slot, which is enough to offset the cost of one extra LLM call.

Reflection has the highest ProxyCost because of two overlapping factors: deep pipeline and long outputs. The three-stage design produces at least three full LLM calls per conversation turn, and the Reflection step tends to add more content than the draft rather than making focused fixes. Token usage reaches 4.4 times that of PlanExecute, and this is a

fixed structural cost built into the design that cannot be removed by adjusting prompts or settings.

ReAct shows an interesting cost pattern: its token count is lower than Single-slot, but its ProxyCost is higher. This is because ProxyCost gives 30% of its weight to LLM call count as a separate factor. In the Thought-Action loop, each Action step triggers one separate LLM call, and the call count builds up across multiple iterations, raising the total cost. The savings on tokens cannot fully cancel out the higher call count. In real API use, call count is usually billed alongside token count, so ReAct's actual cost pressure is higher than its token numbers suggest.

In OOS, NetValue falls by about 60% across all architectures while token cost rises only slightly. Quality and cost are driven by completely different forces: the small cost increase comes from a bit of extra exploration when facing unknown intents, while the quality drop reflects the combined effect of gaps in tool coverage and the model's reasoning limit, which no architecture can bypass. As a result, the only action that can actually improve OOS NetValue is expanding the tool set, not changing the architecture. For e-commerce platforms, this means that when intent coverage is limited, investing in expanding the tool set (such as adding support for holiday promotions, shipping problems, or customer complaints) will improve service outcomes more than upgrading to a more complex architecture.

5.3 Marginal Benefit Summary: Implications for Evaluation and Deployment

Looking at ΔMB, which combines quality and cost, PlanExecute is the only architecture with positive values in both IS (ΔMB = +0.162) and OOS (ΔMB = +0.125). Its positive ΔMB comes from dominating on two dimensions at once: it has both the highest quality and the lowest cost — the planning step reduces execution complexity and lowers token usage, while the execution step's format discipline improves tool call accuracy and raises quality. Because PlanExecute leads on both dimensions at once, its ΔMB ranking stays fully stable across all valid metric weight combinations. The conclusion does not depend on any specific metric design choice.

All other architectures have negative ΔMB, showing a common problem with complex designs when the underlying model has limited capability. ReAct has the highest S_Outcome, but its quality gain is cancelled out by its high-call-count cost structure (ProxyCost higher than Single-slot). Reflection's 4.4x token cost combined with its self-correction failure means costs far outweigh quality output. The shared root cause is that within the capability range of llama3.1:8b, the model's quality ceiling already controls the final result for all architectures. The design advantages of complex pipelines cannot be fully used, so the small quality gains cannot cover the added costs.

In OOS, the extreme gap between PlanExecute's valid termination rate (96%) and its Judge score (27.3) points to a basic problem in evaluation framework design. Rule-based metrics (valid termination rate) and semantic metrics (Judge score) move together in IS because the tool set is sufficient, but they break apart in OOS when the tool set hits its coverage limit. Valid termination rate only means the system produced some response when the tool set is not sufficient — it does not mean the task was actually solved. This breakdown shows a basic limit of proxy metrics: when a proxy metric and the true goal move together inside the training distribution, the proxy looks reliable, but once the distribution shifts, the relationship disappears. This study's OOS paradox is a concrete example of this problem in LLM Agent evaluation, and it suggests that benchmark designs should include OOS probes to actively test where proxy metrics stop being valid. It should be noted that this

study's OOS sample size is 50 cases, which limits the statistical power for intent-level analysis. These OOS conclusions should be treated as exploratory findings rather than definitive results. In addition, the IS and OOS datasets come from different sources, so the observed performance drop includes both the effect of new intents and the effect of a different data distribution. These two sources cannot be fully separated under the current study design. The OOS conclusions in this study are best understood as a cross-dataset stress test rather than a strict out-of-distribution generalization evaluation.
In summary, this study recommends using ΔMB instead of valid termination rate or a single quality score as the main metric for choosing an architecture. Mature platforms with good intent coverage (such as those that have already defined standard intents for returns, order tracking, and promotions) should use PlanExecute first. Its advantage on both quality and cost holds in both IS and OOS, making it the most cost-effective choice when the LLM budget is limited. In cold-start situations where the intent distribution is not yet clear (such as a new platform launch or a new product category), Single-slot's predictable stopping mechanism provides more stable cost protection and can serve as a fallback in a mixed routing architecture for handling uncertain intents.
In local lightweight model deployments, the design advantages of complex pipelines may not be usable due to the model's capability ceiling, and the ReAct and Reflection cases in this study are concrete examples of this risk. It is important to note that the conclusions of this study are closely tied to the capability of the underlying model. If this study were repeated with a more capable model, the relative performance of ReAct and Reflection might improve significantly, and the architecture ranking could change.

6. Conclusion

This study evaluates four Agentic AI design patterns for e-commerce customer service using a multi-dimensional framework that covers quality, process, efficiency, safety, and cost-benefit performance. The central finding is that Plan-and-Execute achieves double dominance: it has both the highest quality score (S_Agent) and the lowest cost (ProxyCost) among all four architectures, resulting in positive marginal benefit (ΔMB) in both in-sample and out-of-sample settings. This advantage holds across all valid metric weight combinations, showing that the conclusion does not depend on any specific evaluation design choice.
ReAct and Reflection both show negative ΔMB, indicating that more complex pipelines do not always justify their added costs when the underlying model has limited capability. ReAct's tool call format is unstable under accumulated context, and Reflection's self-correction step does not produce quality gains proportional to its token cost. The OOS results reveal a further finding: when the tool set cannot cover new intents, rule-based termination metrics and semantic quality metrics decouple, producing false signals of success. This points to a broader limitation of proxy metrics in LLM Agent evaluation and suggests that benchmark designs should include OOS probes to test where proxy metrics stop being valid.
Based on these findings, this study recommends using ΔMB as the primary metric for architecture selection. Mature e-commerce platforms with good intent coverage should use Plan-and-Execute as the default architecture. Cold-start or uncertain-coverage situations are better served by Single-Slot, which provides predictable stopping behavior and avoids loop failures under unknown intents. Two important limitations apply: the conclusions are closely tied to the capability of llama3.1:8b, and the OOS evaluation is based on 50 cases, which limits statistical power at the intent level. Future work should repeat this evaluation with more capable

models and test whether the double-dominance advantage of Plan-and-Execute holds under different capability conditions.

7. Reference

1. Zhang, Y., et al., *Optimizing AI strategies in e-commerce customer service: An agent-based simulation.* Electronic Markets, 2025. **35**(1): p. 77.
2. Thorat, S.H., *Agentic AI for Customer Service and Contact Center Solutions.* Journal of Computer Science and Technology Studies, 2025. **7**(8): p. 444–451.
3. Huang, L., et al. *Towards agentic AI for science: hypothesis generation, comprehension, quantification, and validation*. in *Companion Proceedings of the ACM on Web Conference 2025*. 2025.
4. Yao, S., et al., *React: Synergizing reasoning and acting in language models.* arXiv preprint arXiv:2210.03629, 2022.
5. Shinn, N., et al., *Reflexion: Language agents with verbal reinforcement learning, 2023.* URL https://arxiv. org/abs/2303.11366, 2024. **8**.
6. Wang, L., et al. *Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models*. in *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*. 2023.
7. Luo, J., et al., *Large language model agent: A survey on methodology, applications and challenges.* arXiv preprint arXiv:2503.21460, 2025.
8. Lu, J., et al. *Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities*. in *Findings of the Association for Computational Linguistics: NAACL 2025*. 2025.
9. Yehudai, A., et al., *Survey on evaluation of llm-based agents.* arXiv preprint arXiv:2503.16416, 2025.
10. Kim, W., et al., *Beyond the Final Answer: Evaluating the Reasoning Trajectories of Tool-Augmented Agents.* arXiv preprint arXiv:2510.02837, 2025.
11. Zhuo, J., et al. *ProSA: Assessing and understanding the prompt sensitivity of LLMs*. in *Findings of the Association for Computational Linguistics: EMNLP 2024*. 2024.